YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT A CONSTITUENT UNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY) BALMATTA, MANGALORE

HIGH LEVEL DESIGN

ON

CLOUD COST INTELLIGENCE FOR HEALTHCARE

SUBMITTED BY

MOHAMMED MUNEEB

REG NO :

23BCAICD063

LH-06

GUIDED BY

MISS SINDHU

SUBMISSION DATE

17-04-2026

Table of Contents

1 Introduction
- 1.1 Scope of the document
- 1.2 Intended Audience
- 1.3 System overview

2 System Design
- 2.1 Application Design
- 2.2 Process Flow
- 2.3 Information Flow
- 2.4 Components Design
- 2.5 Key Design Considerations
- 2.6 API Catalogue

3 Data Design.
- 3.1 Data Model
- 3.2 Data Access Mechanism
- 3.3 Data Retention Policies
- 3.4 Data Migration

4 Interfaces

5 State and Session Management

6 Caching

7 Non-Functional Requirements
- 7.1 Security Aspects
- 7.2 Performance Aspects

8 References

1. Introduction

1.1 Scope of the Document

This document presents the High-Level Design (HLD) of the Cloud Cost Intelligence System for Healthcare. It provides an overview of the system architecture, key components, and the overall flow of data within the system. The purpose of this document is to clearly explain how the system is structured and how different modules interact with each other to deliver meaningful insights from cloud cost data.

1.2 Intended Audience

This document is intended for developers, academic evaluators, and project reviewers who need to understand the overall design and working of the system. It is also useful for anyone interested in understanding how cloud cost analytics can be implemented in a healthcare context.

1.3 System Overview

The Cloud Cost Intelligence System is designed to analyze and visualize cloud usage data in a healthcare environment. It helps in identifying cost distribution across departments and services, detecting unusual spending patterns, and understanding how seasonal factors affect cloud usage.

The system processes input data, performs analysis using statistical methods, and presents the results through an interactive dashboard. It includes multiple modules such as data processing, visualization, anomaly detection, and automated insight generation, all working together to provide a clear and structured view of cloud expenses.

2.System Design

2.1 Application Design

The application follows a layered architecture to ensure clarity and modularity:

- **Data Layer:**
  This layer is responsible for storing the dataset in CSV format. It acts as the input source for the system and contains all relevant information such as cost, department, service type, and seasonal indicators.
- **Processing Layer:**
  This layer performs data cleaning, transformation, and analysis using Python libraries such as Pandas and NumPy. It prepares the data for visualization by aggregating and organizing it based on different parameters.
- **Presentation Layer:**
  This layer is implemented using Streamlit and is responsible for displaying the results in the form of charts, graphs, and insights. It provides an interactive interface for users to explore the data.

2.2 Process Flow

The system follows a structured workflow to process and analyze data:

1. The user uploads a cloud cost dataset in CSV format
2. The system validates the dataset to ensure required fields are present

3. The data is processed and aggregated based on month, department, and service type
4. Cost analysis and anomaly detection are performed using statistical methods
5. Insights are generated automatically based on observed patterns
6. The results are displayed through an interactive dashboard

This step-by-step flow ensures smooth data handling and meaningful output generation.

2.3 Information Flow

The flow of information in the system can be summarized as:

User → File Upload → Data Processing → Analysis → Visualization → Insights

This represents how raw data is transformed into useful information for decision-making.

2.4 Components Design

The system is divided into several functional components:

- **Data Loader:**
  Responsible for reading the dataset from the uploaded CSV file and converting it into a usable format.
- **Data Processor:**
  Handles data cleaning, transformation, and aggregation. It prepares the dataset for analysis.
- **Visualization Module:**
  Generates visual representations such as pie charts, bar graphs, line charts, and scatter plots.
- **Anomaly Detection Module:**
  Identifies unusual patterns in cloud cost data using rule-based statistical methods.
- **Insight Generator:**
  Produces automated insights based on trends, anomalies, and cost distribution.

Each component is designed to perform a specific task, ensuring modularity and clarity in the system design.

2.5 Key Design Considerations

The system is designed keeping the following factors in mind:

- **Simplicity:** The system should be easy to use and understand
- **Scalability:** It should handle larger datasets in the future
- **Clarity:** Visualizations should clearly represent data insights
- **Domain Relevance:** The system should reflect real-world healthcare scenarios

These considerations ensure that the system remains practical and user-friendly.

2.6 API Catalogue

This project does not use any external APIs. All data processing and analysis are performed locally using Python libraries.

3. Data Design

3.1 Data Model

The dataset includes multiple fields such as month, department, service type, cost, and cost components like compute, storage, transfer, and API costs. It also includes seasonal indicators to simulate real-world healthcare demand patterns.

3.2 Data Access Mechanism

The data is accessed using the Pandas library in Python. It is loaded into memory and processed dynamically during runtime.

3.3 Data Retention Policies

The data is not permanently stored in the system. It is only used during the active session of the application and is cleared once the session ends.

3.4 Data Migration

Data migration is not required in this project as it uses a standalone dataset.

4. Interfaces

The system provides a web-based user interface using Streamlit. This interface allows users to upload datasets and interact with visualizations and insights easily.

5. State and Session Management

State and session management are handled internally by Streamlit. The system maintains user interaction and data processing within a single session.

6. Caching

Caching can be optionally implemented using Streamlit features to improve performance, especially when working with larger datasets.

7. Non-Functional Requirements

7.1 Security Aspects

- The system does not handle sensitive or real patient data
- Data is processed locally, ensuring privacy
- No external data sharing is involved

7.2 Performance Aspects

- Efficient data processing using Pandas
- Fast rendering of charts and graphs
- Ability to handle moderately large datasets without performance issues

8. References

1. Pandas Documentation – https://pandas.pydata.org/

2. NumPy Documentation – https://numpy.org/

3. Streamlit Documentation – https://docs.streamlit.io/

4. Scikit-learn Documentation – https://scikit-learn.org/

5. Cloud Computing Concepts – https://aws.amazon.com/what-is-cloud-computing/